AXIOM: A Correctness-Aware Graph-Attention Architecture for Federated Multi-Agent Slice Admission Control

Manoj Prasad Kunasegran[1], Wai Leong Pang[1*], Swee King Phang[1]

[1]School of Engineering, Taylor's University, Subang Jaya, Malaysia.

*Corresponding author(s). E-mail(s): waileong.pang@taylors.edu.my;

Abstract

Fifth generation (5G) network slicing must admit heterogeneous service requests across many cooperating base stations under strict per-slice guarantees. This paper presents AXIOM, a correctness-aware graph-attention architecture for federated multi-agent slice admission control that closes three gaps from our prior systematic review, namely trustworthy evaluation, privacy-preserving coordination, and validated deployment. A shared Graph Attention Network (GAT) encoder reads the base-station cluster graph and feeds decentralised per-slice action heads, so each gNodeB (gNB) trains on a cluster-wide view yet decides locally. AXIOM is the only policy among three that is strong on reward, blocking accuracy, and scalability at once. It earns 11.6% higher mean reward per step than an independent-learner baseline and blocks the correct slice 5.7% more often. Scaled from 3 to 19 gNBs, a flattened single-agent baseline collapses into always-accept in every one of 45 test cells, whereas AXIOM cuts that collapse rate to a stable 35.4% that is already present at 7 gNBs. Federated training adds no measurable reward cost, and differential privacy keeps blocking accuracy perfect up to a clear noise threshold. A correctness-aware metric pair and a reproduction-versus-replication discipline keep every result trustworthy, catching four evaluation-pipeline faults a standard pipeline would have reported as clean. A live deployment on OpenAirInterface (OAI) hardware serves every slice at 0.000% loss and executes the offline-trained logic correctly end to end, establishing deployability on a real Open Radio Access Network (O-RAN) stack. Contention-dependent behaviour is characterised in a disclosed offline stress regime calibrated harder than the live rig reaches.

Keywords: Network Slicing, Slice Admission Control, Multi-Agent Reinforcement Learning, Graph Attention Networks, Federated Learning, Open RAN

M. P. Kunasegran: orcid.org/0009-0005-3169-1873. W. L. Pang: orcid.org/0000-0001-8407-5648. S. K. Phang: orcid.org/0000-0002-7877-8766.

1 Introduction

TODO(MANOJ): The abstract and the introduction's result sentences still quote old-environment numbers. Rewrite both once all shared-budget reruns finish.

Network slicing virtualises the physical radio access network (RAN) to serve heterogeneous services under Ultra-Reliable Low-Latency Communication (URLLC), enhanced Mobile Broadband (eMBB), and massive Machine-Type Communication (mMTC) categories. Each slice carries distinct Service Level Agreement (SLA) targets that are in turn based on Quality of Service (QoS) thresholds. Our earlier work [8, 9] validated that a Deep Q-Network (DQN) admission controller can enforce SLAs for a single gNodeB (gNB) in a live OpenAirInterface (OAI) testbed. However, a real deployment spans multiple gNBs that share backhaul, spectrum planning, and a cluster-wide load-balance objective. Thus, slice admission at one gNB is rarely isolated, and coordinated admission is in fact a multi-gNB, multi-agent challenge. This research aims to address whether a shared, graph-structured view of the cluster could enable per-gNB agents to decide better than a single agent on a flattened state, as opposed to independent agents. Fig. 1 places the research premise in context.

This paper introduces AXIOM, an Attention-based eXecution with Integrity for O-RAN Multi-agent slice-admission controller. AXIOM is a centralised-training, decentralised-execution multi-agent architecture that is built on a shared Graph Attention Network (GAT) [23] encoder over a multi-gNB topology, which feeds decentralised per-slice Q-heads. We deliver it together with the evaluation discipline needed to trust what it reports. Three gaps from our systematic review of 67 core studies motivate this pairing, as Section 2 details. First, no reviewed study checks whether its own outcome metric can score a degenerate policy as well as a correct one. Second,

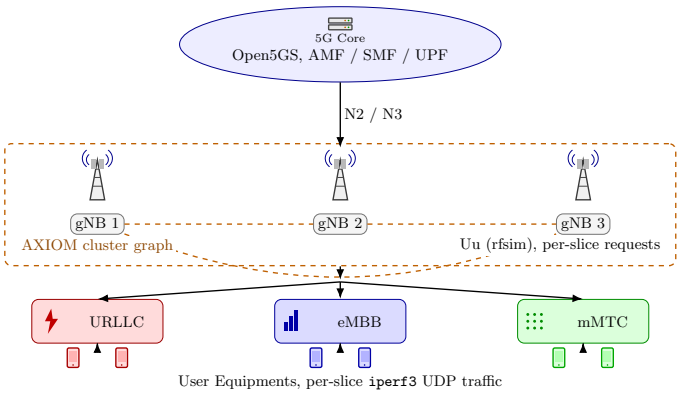


Fig. 1 System view. A cluster of gNBs, coordinated by the shared AXIOM encoder over the cluster graph and backhauled to a 5G core, serves UEs across the URLLC, eMBB, and mMTC slices. Each gNB admits or rejects per-slice requests through a PRB-ratio ceiling.

none replicates a headline architectural finding on an independent seed sample. Third, none tests whether a live testbed can reach the contention regime its offline claims assume. We close all three gaps through a per-component realisation of the architecture, namely the GAT encoder, the Multi-Agent Reinforcement Learning (MARL) controller, the Federated Learning (FL) coordinator, and the Differential Privacy (DP) layer, each measured and isolated rather than folded into one end-to-end number. We apply a correctness-aware metric pair and a reproduction-versus-replication discipline throughout, and we add a live single-gNB deployment that tests, rather than assumes, whether the rig reaches genuine per-slice scarcity.

We extend the architecture to federated training with differential privacy, and we stress it under mid-episode gNB dropout, demand spikes, agent churn, and cluster growth to $N \in \{7, 19\}$ gNBs under ring and hex-grid adjacency. On the objective it trains on, AXIOM decisively beats an independent-learner ablation and resists a training collapse that becomes total for a flattened single-agent baseline. A correctness-aware metric pair and an independent-seed replication keep every reported result trustworthy.

2 Related Work

Our prior systematic review [10] screened 6,286 records down to 67 core studies from 2024 to 2026 on machine-learning-driven, user-centric distributed network slicing. The review organised them around three enabling pillars, namely topology-aware graph representation, distributed multi-agent control, and privacy-preserving federation. Each pillar is well developed on its own, but the three are rarely combined under realistic constraints, and never with a validated live deployment. No reviewed study checks whether its own evaluation metric can mask a degenerate policy, and none replicates a headline finding on an independent seed sample, which is the gap this paper fills.

AXIOM also sits inside the broader Open RAN (O-RAN) effort to run learned control loops as applications over standard interfaces. Polese et al. [15] survey that architecture and its open problems, while ColO-RAN [14] and Tsampazi et al. [22] train and compare deep reinforcement learning controllers for slicing on large experimental platforms. Learned slice admission predates O-RAN, from market-driven admission of slice requests [3] to reinforcement learning for RAN slicing [18]. For cooperating agents, value factorisation such as QMIX [17] and centralised-critic policy gradients such as MAPPO [25] are the standard training schemes. AXIOM instead shares one graph encoder across agents and trains every pending request as its own temporal-difference sample, for the reason Section 4.1 gives.

Several recent works come close on a single axis of this problem. Shao et al. [19] motivate our GAT encoder choice by showing that graph attention strengthens inter-agent cooperation over plain DQN and Advantage Actor-Critic (A2C) baselines. Ahmadi et al. [2] study topology generalisation through graph attention, which complements but does not answer the fixed-topology evaluation-integrity question we ask in Section 5.5. Sulaiman et al. [20] are the closest prior art to our joint slicing-and-admission-control problem. None of these three checks whether its reward-optimal

policy could be mistaken for a degenerate one, and none replicates a headline result on an independent seed sample.

Robustness studies share our concern but stop short of it. Tashman and Cherkaoui [21] and Fatehi et al. [6] study robustness to a disruptive condition, as our disruption campaign in Section 5.4 does, yet neither checks whether the reported robustness metric is trustworthy and neither replicates a finding on an independent sample. That is the gap our independent-seed replication closes, and the need is concrete, since replication overturned our initial churn-immunity reading. Zhang et al. [26] are the closest periodic-synchronisation precedent, and Yasin et al. [24] are the closest precedent for federated learning with differential privacy. Their threat model differs from ours, since they secure the controller rather than characterise a control policy's own privacy-utility trade-off, which our σ sweep in Section 5.3 reports, and neither adds a correctness-aware check on whether federating or privatising changed decision quality. A further, closely related set [4, 7, 13, 16] combines two of the three pillars, yet like every work above, each evaluates on its own reported outcome metric, computed once, on one seed or run.

Rather than adding a new point in the GAT, MARL, and FL design space, this paper fills that gap with a demonstration on AXIOM, extended to federation, privacy, and disruption. It shows that two checks are both necessary, namely testing a metric's own trustworthiness and replicating a finding on an independent sample. Skipping either changes what a reader may believe the resulting numbers mean.

3 System Model and Problem Formulation

Fig. 2 provides an overview of the system. The AXIOM controller runs across four offline campaigns in a disclosed stress environment. Two correctness safeguards check every result. The deployment pipeline is then checked on the live single-gNB testbed of Fig. 3.

We consider a cluster of N=3 gNBs, each serving the same three slices, URLLC, eMBB, and mMTC, under a shared per-gNB Physical Resource Block (PRB) budget B. An admission decision is a PRB-ceiling adjustment through the `slicing_control_m` minimum and maximum ratio, rather than a literal per-flow accept or reject, since the real OAI xApp control interface exposes only ratio ceilings. No real multi-gNB physical topology is available to us, so we take the least assumption-laden choice and treat the cluster as fully connected, as a modelling choice rather than a measured fact. At each control step, every gNB holds zero or more pending admission requests, each tagged with a slice identity, and an agent decides whether to accept or reject for each request independently, conditioned on the current cluster state. The discrete action space matches our single-gNB work exactly at `action_dim`=2, so any coordination benefit found here is due to the architecture and not a richer action space.

The step reward follows our prior single-gNB formulation. It has a per-slice service term net of SLA-violation penalties, a congestion charge on admitted volume, and a load-imbalance penalty across gNBs. The step reward r_t sums over slices indexed by k. It uses priority weight ω_k, per-accept reward R_k, admitted count n_k, violation penalty λ_k, and the indicator $\mathbf{1}[\cdot]$ equal to one exactly when slice k is in SLA violation

that step,

$$r_t = \sum_k \big(\omega_k R_k\, n_k - \lambda_k\, \mathbf{1}[v_k]\big) \;-\; \kappa\, C_t \|a_t\|_1 \;-\; \beta\,(1 - F_t), \tag{1}$$

where $\|a_t\|_1$ is the total admitted this step, C_t the cluster-mean congestion level, κ the congestion coefficient, $F_t \in [0, 1]$ the min-over-max PRB-utilisation fairness ratio, and β the load-balance weight. The violation flag v_k follows from a continuous SLA margin m_k. The correctness-aware metrics of Section 4.4 also build on this margin. We take it as the worse of the queue and loss channels,

$$m_k = \min\Big(1 - \tilde{q}_k,\; 1 - \tfrac{\ell_k}{\ell_k^{\max}}\Big), \qquad v_k \Leftrightarrow m_k \le 0, \tag{2}$$

with $\tilde{q}_k$ the slice backlog relative to its budget, and ℓ_k and $\ell_k^{\max}$ its loss proxy and loss budget. The weights are set so that, under congestion, the intended policy is differentiated shedding, which keeps accepting URLLC and eMBB and sheds mMTC.

Every result uses an offline, closed-loop simulation of the cluster, except the bounded live anchor of Section 5.6. Each gNB has a budget of B=100 PRBs shared by its three slices. Each slice's mean offered demand is 1.05 times its nominal PRB share of 30, 60, and 10 PRBs for URLLC, eMBB, and mMTC, so aggregate demand exceeds the budget by about five per cent while URLLC and eMBB demand stays below their own caps of 33 and 66 PRBs. Whenever the slices' admitted demand exceeds B, they share B in proportion to that demand, so every admitted mMTC request takes resources from URLLC and eMBB. Unserved demand carries forward as backlog, which drives the SLA margin of Eq. (2), and per-gNB load multipliers drawn from $[0.9, 1.0]$ keep the cluster heterogeneous. This shared-budget design replaces an earlier simulator that served each slice independently at about a quarter of the budget,

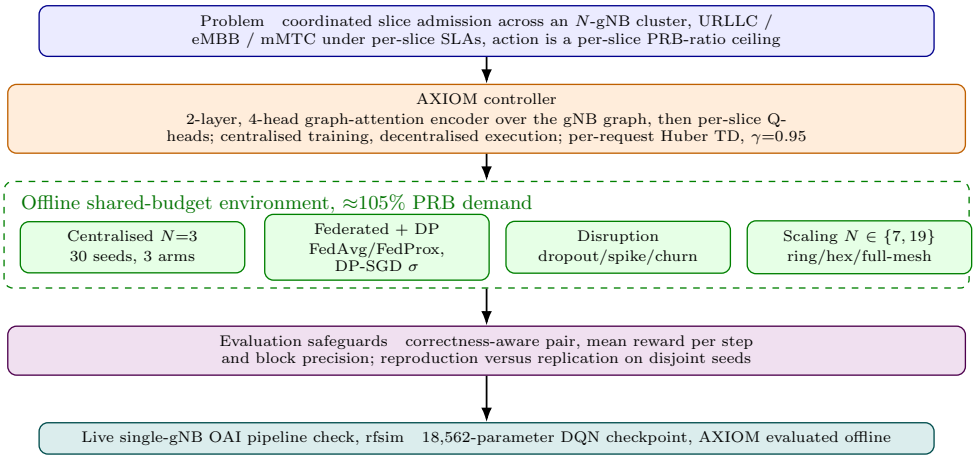


Fig. 2 Overview of the research. An AXIOM controller is evaluated across four offline campaigns in a stress environment, every result checked by two safeguards, with the deployment pipeline checked on a live single-gNB testbed.

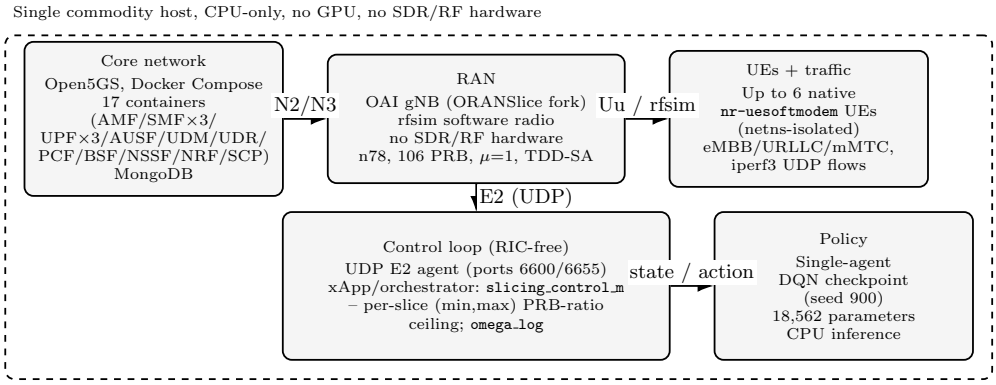


Fig. 3 Real testbed stack, a single-host, software-radio rfsim, RIC-free O-RAN deployment distinguished from SDR hardware-in-the-loop testbeds. Every block and label is a measured or configured value from this rig.

where accepting every request was harmless. Section 4.4 shows how a trivial accept-all baseline exposed that, and verifies that shedding mMTC is both the reward-optimal and the SLA-protecting behaviour in the current design. The concrete rig behind every live measurement is the single-host, software-radio, O-RAN stack of Fig. 3, which runs without a near-real-time RAN Intelligent Controller (RIC). We omit the RIC because the ORANSlice fork applies slicing control directly through a lightweight UDP E2 agent, which exercises the real per-slice PRB-ratio primitive without the overhead of a separate RIC on a single commodity host.

4 The AXIOM Architecture

AXIOM differs from its two baselines in exactly what each policy sees and shares. AXIOM runs one graph-attention encoder over the whole gNB cluster, trains that encoder jointly from every agent's experience, and reads out one dedicated Q-head per slice, so every gNB decides from a shared, topology-aware view of the cluster. The single-agent DQN keeps one controller over a flattened joint state, with no per-gNB structure and no graph. The independent DQN gives each gNB its own learner that sees only local features and never shares parameters or reads the cluster topology. The two baselines are therefore ablations of AXIOM that remove the shared encoder and the graph input in turn, so any gap between them and AXIOM measures what those two components add.

4.1 Shared Encoder, Decentralised Q-Heads

Fig. 4a shows the architecture. At every control step, each gNB forms its own per-slice feature row x_i. The features are PRB-used-ratio, congestion-level, and queue-length-norm. These rows stack into a joint node-feature matrix $X \in \mathbb{R}^{N \times d}$ over the N gNB nodes, with d features each. A two-layer, H=4-head GAT [23] produces one embedding per gNB. Each head h decomposes additive attention rather than scoring

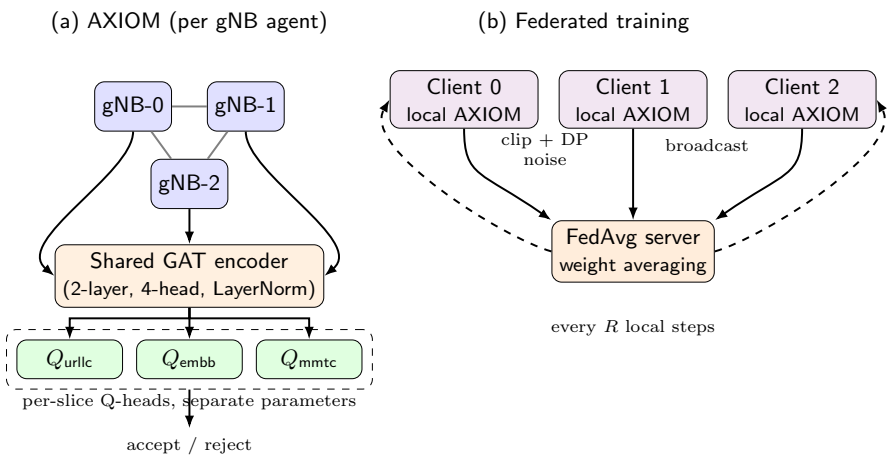


Fig. 4 AXIOM architecture. a, A shared graph-attention encoder over the fully-connected 3-gNB topology produces one embedding per node, consumed at execution time by per-slice Q-heads using only that node's row. b, The federated variant, each gNB a client synchronised by periodic FedAvg with an optional DP-SGD clip-and-noise step.

a concatenation. It uses a learned source vector a_s^h and destination vector a_d^h, applied to the shared linear projection W^h. Here ϕ is the leaky rectifier of slope 0.2, and $\mathcal{N}_i$ is the set of neighbours of node i under the adjacency mask,

$$\varepsilon_{ij}^h = \phi\big(a_s^{h\top} W^h x_i + a_d^{h\top} W^h x_j\big), \qquad \alpha_{ij}^h = \frac{\exp \varepsilon_{ij}^h}{\sum_{l\in\mathcal{N}_i} \exp \varepsilon_{il}^h}. \tag{3}$$

Here ε_{ij}^h is the unnormalised attention score from node i to neighbour j in head h, and α_{ij}^h is its normalisation over i's neighbours, with l ranging over $\mathcal{N}_i$. Hidden layers concatenate the H heads under an ELU nonlinearity, while the final layer averages them. A per-layer normalisation operator $\mathrm{LN}(\cdot)$ then rescales each node's embedding to zero mean and unit variance, and produces the final per-gNB embedding z_i,

$$z_i = \mathrm{LN}\Big(\tfrac{1}{H}\textstyle\sum_{h=1}^{H} \sum_{j\in\mathcal{N}_i} \alpha_{ij}^h\, W^h x_j\Big). \tag{4}$$

Centralisation comes from this encoder, whose parameters every agent's loss updates jointly, so it sees the full cluster state during training. Decentralised execution comes next, since at action-selection time each agent consumes only its own embedding row z_i to decide its own pending requests.

The `LayerNorm` and the per-slice heads are deliberate design choices. With a single shared Q-head, all three slices update the same parameters, and URLLC's large priority weight produces gradients that swamp mMTC's much smaller signal, a standard shared-parameter class imbalance. One small multi-layer perceptron (MLP) per slice, selected by the context one-hot, gives each slice its own parameters, and the `LayerNorm` stops an embedding from encoding congestion through its magnitude alone. A seed

Algorithm 1 AXIOM: centralised training, decentralised execution

Require: encoder f_θ, per-slice Q-heads $\{Q_{\phi_k}\}$, targets $f_{\theta^-}, \{Q_{\phi_k^-}\}$, buffer $\mathcal{D}$, adjacency A, discount γ, sync period C

for each step t, observing joint node features X_t **do**
  $E_t \leftarrow f_\theta(X_t, A)$ {centralised, one row per gNB}
  for each pending request at gNB i, slice k, pick $a \leftarrow \arg\max_a Q_{\phi_k}(E_t[i])[a]$, ϵ-greedy {decentralised, own row only}
  step env, store $(X_t, X_{t+1}, \{(i,k,a)\}, r_t, \text{done})$ in $\mathcal{D}$
  if $|\mathcal{D}| \geq$ warmup **then**
    sample minibatch B; $E_B \leftarrow f_\theta(X_B, A)$, $E'_B \leftarrow f_{\theta^-}(X'_B, A)$
    per request, $y \leftarrow r + \gamma(1-\text{done})\max_{a'} Q_{\phi_k^-}(E'_B[i])[a']$
    $\mathcal{L} \leftarrow$ mean per-request Huber loss, never summed per step; update $\theta, \{\phi_k\}$, clip grad-norm 1.0
  end if
  every C episodes, $\theta^- \leftarrow \theta$, $\phi_k^- \leftarrow \phi_k$
end for

here is one independent random initialisation of network weights and environment stream, and we report every seed rather than a single best run.

Algorithm 1 states the training loop, where the encoder updates from every request in a sampled minibatch regardless of gNB or slice. This is centralised training, while action selection at rollout time consults only the acting agent's own embedding row, which is decentralised execution. Every pending request at gNB i, slice k is its own temporal-difference sample, and we never sum requests across a step before the loss. For a request with immediate reward r, next-state embedding z'_i, and candidate next action a', the bootstrapped target y at discount γ and the minibatch loss are

$$y = r + \gamma\,(1-d)\max_{a'} Q_{\phi_k^-}(z'_i)[a'], \qquad \mathcal{L} = \frac{1}{|B|}\sum_{(i,k,a)\in B} \ell_\delta\Big(Q_{\phi_k}(z_i)[a] - y\Big), \tag{5}$$

where Q_{ϕ_k} is slice k's online Q-head, $Q_{\phi_k^-}$ its target copy, d the episode-done flag, and B the sampled minibatch of requests. We use the Huber loss ℓ_δ in place of squared error, which keeps the deeper encoder stable under long bootstraps. An earlier variant passed each gNB's summed per-request values through a QMIX-style monotonic mixer [17]. Because the number of pending requests changes from step to step, that mixed target changed scale with the request count, and the mean training loss diverged, rising from about 76 over the first fifth of training steps to about 409 over the last fifth. Treating each request as its own sample removes that dependence.

4.2 Federated Training with Differential Privacy

Fig. 4b shows the federated variant. Each gNB becomes a federated-learning client. It holds a private copy of the same network and trains only on its own requests. Transitions are partitioned by owning agent, so no client sees another's raw admission data. Each client's GAT encoder still consumes the full joint node-feature snapshot.

Every R local steps, the server averages all clients' online weights by FedAvg [12] and broadcasts the result back. We also implement FedProx [11], which adds a proximal term that pulls local weights toward the last broadcast global model. We sweep it at $\mu \in \{0.01, 0.1, 1.0\}$ under both homogeneous and heterogeneous per-gNB load. FedProx earns no measurable dividend at any strength, since per-round client drift is negligible relative to the loss scale.

Differential privacy gives a formal guarantee that one client's training data cannot be inferred from the shared model. We apply it as a standard differentially private stochastic gradient descent (DP-SGD) step [1]. Each client clips its gradient to a fixed norm C_{dp}. When privacy is enabled, the client then perturbs the gradient with Gaussian noise scaled by a noise multiplier σ, before its optimiser step. We report σ as the primary privacy parameter. Alongside it we report a formal (ε, δ) bound via zero-concentrated differential privacy (zCDP) [5]. Here ε and δ are the standard privacy-loss and failure-probability parameters. Composing the Gaussian mechanism over m per-client gradient releases gives the zCDP parameter ρ and the bound

$$\rho = \frac{m}{2\sigma^2}, \qquad \varepsilon(\delta) = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{6}$$

We apply the same clip whether or not noise is added, so a σ=0 run and a σ>0 run differ only in whether noise is injected, which is what lets the privacy-cost comparison in Section 5.3 isolate the effect of the noise. Each federated round therefore runs R local Huber-TD steps per client with an optional FedProx proximal term $\frac{\mu}{2}\|\theta_i - \bar{\theta}\|^2$, clips and optionally perturbs the gradient, and closes with a FedAvg average that is broadcast back.

4.3 Experimental Setup

Every campaign uses 300 training and 50 evaluation episodes per seed, and each seed is evaluated on a held-out environment stream. The centralised campaign uses 30 seeds, 900 to 929, and compares AXIOM with two learned baselines and one rule. The single-agent DQN is one controller over the flattened joint state. The independent DQN gives each gNB its own DQN with no parameter sharing and no topology information. An ablation is a deliberately reduced variant with one component removed, so its effect can be isolated, and both DQN policies are ablations in that sense.

The oracle rule needs no learning, since it simply accepts every URLLC and eMBB request and sheds mMTC whenever its gNB's congestion level reaches a threshold τ. We choose τ on the training-seed streams only and evaluate it on the held-out streams, and the selected value is τ=0, so the rule sheds every mMTC request. It encodes exactly the behaviour the reward was designed to favour, so it serves as a reference ceiling rather than a competitor. Two further arms isolate AXIOM's components, namely the two DQN baselines rerun at AXIOM's learning rate, and an AXIOM variant whose graph encoder is replaced by a per-node MLP of the same width that ignores the adjacency.

All arms share the discount factor $\gamma = 0.95$, the per-episode epsilon schedule, the target-synchronisation period, and the training budget. AXIOM trains at learning rate $1{\times}10^{-4}$ and the DQN baselines at $1{\times}10^{-3}$ by default, all under the Adam optimiser,

with epsilon decaying from 1.0 to 0.05 at a per-episode factor of 0.985 and the target network synchronised every 10 episodes. The GAT encoder uses hidden and output dimension 16, feeding per-slice Q-heads of hidden dimension 32. A sensitivity study repeats the learned arms and the oracle rule on 10 seeds under three changed conditions. Two change the load, to 1.02 and 1.07 times the nominal shares, and the third perturbs the weights, lowering eMBB's priority weight from 3.5 to 2.5 and raising mMTC's from 0.3 to 0.6.

The federated campaign uses seeds 900 to 909 and a five-level noise sweep $\sigma \in \{0, 0.5, 1, 2, 4\}$, and each federated round runs 50 local steps under FedAvg with a gradient-clip norm of 1.0. A disjoint replication sample, seeds 1000 to 1009, repeats the headline comparisons. All confidence intervals (CIs) are 95% bootstrap percentile intervals over 10,000 resamples. Paired comparisons use the Wilcoxon signed-rank test, and each family of comparisons against AXIOM is Holm-corrected for multiple testing.

4.4 Evaluation Safeguards

Three safeguards make the results below trustworthy, and Table 1 lists the failure each one caught during this work. A standard pipeline would have reported every one of them as a clean result.

The first is a trivial-policy check, in which we evaluate the simplest rules the environment admits before trusting any learned policy, namely accepting every request and shedding one slice outright. In the first version of our simulator, every slice was served independently against its own ceiling at about a quarter of the gNB's budget. There, accepting every request earned 14.45 reward per step and kept URLLC and eMBB at 100% SLA compliance, more than any learned policy, so admission control had nothing to do and policies that appeared to fail by never blocking were in fact optimal. The shared-budget simulator of Section 3 removes that loophole. Over the same 30 held-out seeds, accepting every request now earns -4.05 reward per step and drops URLLC and eMBB compliance to 2.4% and 1.1%, whereas shedding mMTC earns 10.94 and keeps them at 92.2% and 95.6%. Every result in this paper comes from the shared-budget simulator. Its feasible range has an edge, since at 1.07 times the nominal shares even shedding every mMTC request leaves URLLC and eMBB compliance below 7%.

The second is a correctness-aware set of metrics, computed from data every run already logs. Mean reward per step is the training objective. Block precision is the fraction of blocked requests that correctly target mMTC, the slice the reward calibration makes reject-optimal. Writing b_{mMTC} and b for the mMTC-targeted and total blocks over an evaluation, it is $\pi = b_{\mathrm{mMTC}}/b$ for $b > 0$ and undefined for $b = 0$, so a seed that never blocks is counted as a collapse rather than scored as a success. Per-slice SLA compliance then shows whether the shedding actually protects URLLC and eMBB. We do not use the common all-slice compliance metric, because correct shedding gives up mMTC's own SLA by design, so that metric scores a correct policy and a failed one alike.

The third is a reproduction-versus-replication discipline, in which we recompute every headline comparison on an independent, disjoint seed sample and report both.

Table 1 Evaluation failures this paper's safeguards caught during the work, none of which a standard pipeline would have surfaced.

Failure	Missed by	Caught by	Outcome
Inert simulator (§3)	Learned-policy metrics	Accept-all baseline	Shared-budget rerun
No-op scored as fine	All-slice compliance	Precision, per-slice SLA	Collapse made visible
Repro. gap (§5.4)	Single-sample claim	Repro-vs-replication	Churn claim retracted
Scheduler floor (§5.6)	Cross-layer floor	Direct instrumentation	100k+ readings clear

Reproduction re-runs the same seeds, whereas replication draws fresh ones, so only replication can catch a result that is an artifact of one seed sample.

5 Results

We compare AXIOM with two learned baselines and one rule throughout this section. AXIOM is the architecture of this paper, a shared graph-attention encoder that reads the whole base-station cluster and feeds one small decision head per slice. The single-agent DQN flattens the cluster into one controller that sees no per-gNB structure. The independent DQN gives each gNB its own learner that never looks at its neighbours. The two DQN policies are deliberate ablations, so any gap between them and AXIOM isolates what the shared encoder and joint training actually add. The oracle rule, which simply sheds every mMTC request, marks the reward a policy can reach when it knows the intended behaviour in advance.

The comparison rests on four quantities, the first being mean reward per step, the objective the policies train on, so higher is better. Block precision is the share of a policy's rejections that land on the slice the reward rule marks lowest priority, so a value near one means the policy sheds the right traffic. Per-slice SLA compliance shows whether that shedding actually protects URLLC and eMBB. Collapse names a failure mode in which a trained policy stops discriminating and simply accepts every request, which in the shared-budget environment starves the priority slices. Table 2 sets out how the three policies stand on these axes, and the subsections that follow give the evidence.

TODO(MANOJ): Rewrite this headline paragraph and the 19-gNB and federated rows of Table 2 once the shared-budget scaling and federated reruns finish.

The headline at three gNBs is that AXIOM is the only learned policy that reaches the oracle, shedding exactly the right traffic in every seed while both baselines leave the priority slices poorly protected.

5.1 Centralised

The three-gNB fully-connected setting is deliberately the hardest case for a graph encoder, since every gNB already sees every other and there is little topology left to exploit. The encoder's coordination advantage is tested in earnest only once the cluster grows and its adjacency thins, in Section 5.5.

Table 3 and Fig. 5 report the thirty-seed campaign. AXIOM learns the intended policy in every seed. It never collapses, every one of its blocks lands on mMTC, and it keeps URLLC and eMBB at 91.9% and 95.5% SLA compliance. Its mean reward of 10.892 per step reaches 99.6% of the oracle rule's 10.935, and the two tie outright in 28 of the 30 seeds, so AXIOM recovers the reward-optimal behaviour from the reward alone, without being told which slice to shed. The two DQN baselines fall far short, with the single-agent DQN reaching 35.6% of the oracle reward and the independent DQN 15.5%. Both let URLLC and eMBB compliance fall to between 25.5% and 61.6%, because they also turn away some URLLC and eMBB requests and shed mMTC less consistently. AXIOM leads the single-agent DQN by +6.997 per step, with 95% CI [5.01, 9.12], and the independent DQN by +9.199, with 95% CI [7.21, 11.10], both at Holm-adjusted $p < 0.0001$. It wins or ties in all 30 seeds against the single-agent DQN and in 29 of 30 against the independent DQN.

Table 2 How the three policies compare. AXIOM is the architecture of this paper; the two DQN policies are ablations that strip out its coordination. The three-gNB values come from the campaign of Table 3.

Property	AXIOM	Single-agent DQN	Independent DQN
Design	Shared encoder, per-slice heads	One flattened controller	One learner per gNB
Reward/step, 3 gNBs	10.89, oracle-level	3.90	1.69
Block precision, 3 gNBs	1.000	0.877	0.937
URLLC/eMBB SLA, 3 gNBs	91.9%/95.5%	61.6%/31.6%	41.4%/25.5%
Behaviour at 19 gNBs	Partial, 35.4%	Total, all 45 cells	Rare, 0.0–16.7%
Reads cluster layout	Yes	No	No
Federated, privacy-ready	Yes, no reward cost	Not applicable	Not applicable

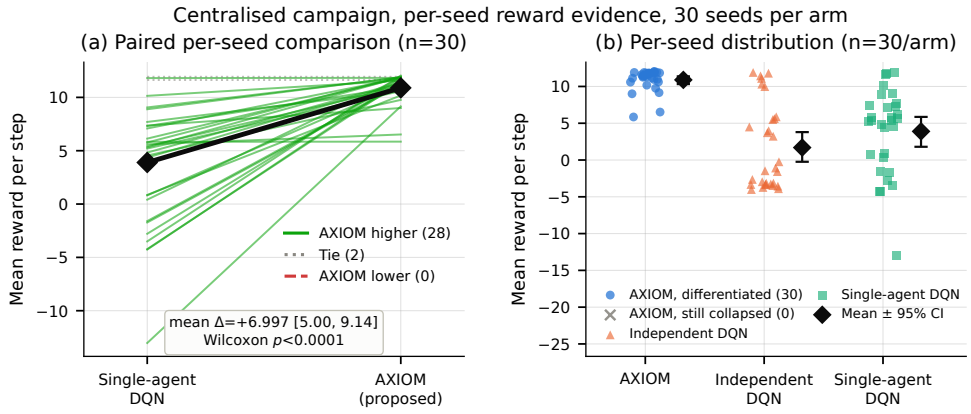


Fig. 5 Centralised campaign, per-seed evidence. a, Paired per-seed mean reward per step of AXIOM versus single-agent DQN, coloured by win, tie, or loss. b, Per-seed reward distribution for the three learned arms.

Table 3 Centralised campaign, 30 seeds per arm, 95% bootstrap CIs. Precision averages the seeds that block at all. SLA columns give per-slice compliance.

Arm	Reward/step	Precision	Collapsed	URLLC	eMBB
Oracle rule	10.935 [10.36, 11.42]	1.000	0/30	0.922	0.956
AXIOM	10.892 [10.32, 11.37]	1.000	0/30	0.919	0.955
AXIOM, MLP encoder	10.723 [9.93, 11.33]	0.999	0/30	0.905	0.951
Single-agent DQN	3.895 [1.74, 5.84]	0.877	0/30	0.616	0.316
at LR 10^{-4}	4.271 [2.04, 6.52]	0.988	0/30	0.544	0.434
Independent DQN	1.693 [−0.22, 3.73]	0.937	1/30	0.414	0.255
at LR 10^{-4}	3.436 [1.31, 5.58]	0.974	0/30	0.480	0.402

5.2 Baselines, Ablations, and Sensitivity

Three checks test where AXIOM's advantage comes from. First, the learning rate does not explain it. Rerun at AXIOM's rate of 10^{-4}, the single-agent DQN reaches 4.271 and the independent DQN 3.436 reward per step, and AXIOM still leads them by +6.621 and +7.457 at Holm-adjusted $p < 0.0001$.

Second, at three fully-connected gNBs the graph encoder adds nothing measurable. Replacing it with a per-node MLP of the same width lowers reward only from 10.892 to 10.723, a difference of +0.169 with 95% CI [−0.04, 0.50] and $p = 0.25$. This is expected, since each gNB's own congestion carries most of this decision, so at this scale the gain comes from the shared parameters, the per-slice heads, and the joint training rather than from attention. Section 5.5 tests whether attention matters once the cluster grows and its adjacency thins.

Third, the advantage holds when the environment changes within its feasible range, and AXIOM tracks the oracle rule under every change, tying it in 7 to 9 of 10 seeds with no significant difference. With demand lowered to 1.02 times the nominal shares, AXIOM keeps URLLC and eMBB at 100% and leads the single-agent and independent DQN by +1.880 and +3.922 reward per step. With eMBB's weight lowered to 2.5 and mMTC's raised to 0.6, it keeps both priority slices above 95% while the baselines let eMBB compliance fall to 1.5% or less, and its lead widens to +5.599 and +9.045. All four leads hold at Holm-adjusted $p < 0.02$ over 10 seeds. At 1.07 times the nominal shares the environment passes its feasibility edge, since even the oracle rule leaves URLLC and eMBB below 7%. There AXIOM matches the oracle, and the single-agent DQN, which also turns away some URLLC and eMBB traffic, keeps more URLLC at 26.4% against 5.7%, with no significant reward difference. Protecting URLLC beyond that edge needs a policy that also sheds some eMBB, which this reward calibration does not encourage.

5.3 Federated Training and DP

TODO(MANOJ): Old-environment numbers. Replace with the shared-budget federated and noise-sweep results, and report the epsilon bound per noise level, when the rerun finishes.

With federation switched on and no privacy noise, mean reward per step barely moves against the centralised run on the same ten seeds, by +0.133 with 95% CI

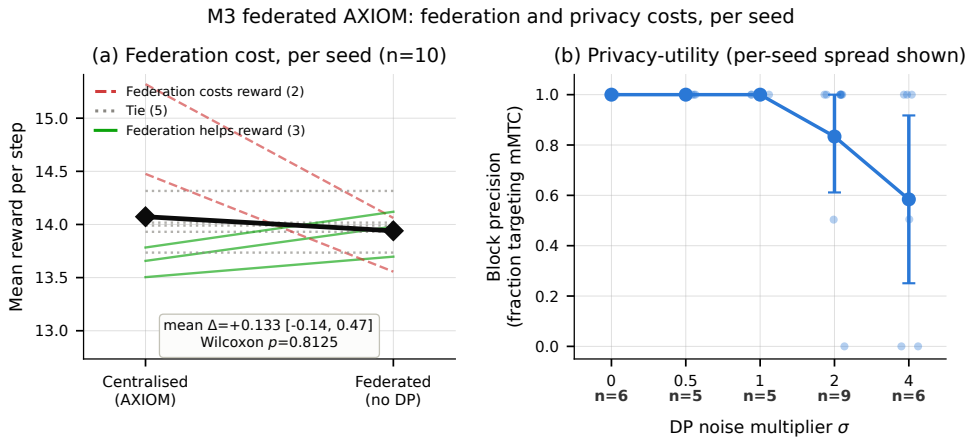


Fig. 6 M3 federated and DP sweep. a, Paired per-seed federation cost on mean reward per step, no measurable difference at $p = 0.8125$. b, Block precision versus DP noise multiplier σ, perfect through σ=1.0 then dropping sharply.

$[-0.138, 0.470]$ and $p = 0.8125$. Federating the training therefore costs nothing measurable, and both variants keep perfect block precision on the seeds that block at all. We then add differential-privacy noise, random perturbation to each shared update that hides any single gNB's data. Block precision holds perfect through a noise multiplier of σ=1.0, then falls sharply to 0.834 at σ=2.0 and 0.584 at σ=4.0, as Fig. 6b shows. This is a clear threshold rather than a gentle slide. Mean reward per step stays inside a narrow 12.45–14.19 band throughout, because the noise erodes which slice gets blocked long before it erodes whether the policy decides at all. Read together with the FedProx null result, this answers a deployability question directly. Periodic, infrequent synchronisation is enough for this architecture, a disaggregated controller needs no per-step coordination between gNBs to match centralised reward, and privacy can be bought at a bounded and known precision cost.

5.4 Disruption

TODO(MANOJ): Old-environment numbers. Replace with the shared-budget disruption results and their replication when the rerun finishes.

We test every policy's frozen checkpoint against three mid-episode disruptions, injected at evaluation time with no retraining. A 'gNB dropout' forces one gNB to reject everything and blanks its row in the shared feature matrix for a window. A 'demand spike' briefly multiplies the arrival rate from outside the policy. An 'agent churn' swaps one gNB's decisions for a fresh, untrained copy of its own policy for a window and leaves the others intact. Churn applies to every genuinely multi-agent policy but not to the single-agent one. For dropout and churn the severity is how long the window lasts, at 10%, 30%, or 60% of the episode. For the spike it is the arrival multiplier, at 2×, 4×, or 8×. Because raw reward grows mechanically with

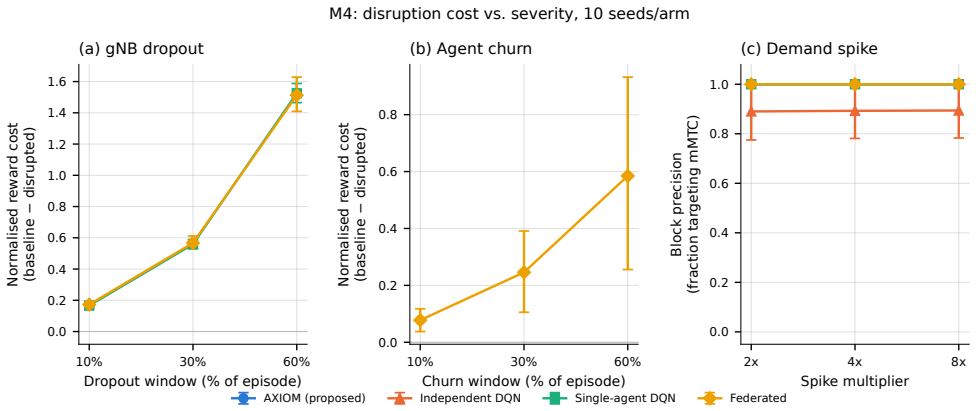


Fig. 7 M4 disruption campaign, 10 seeds per arm. a, b, Normalised reward cost versus severity for dropout and churn, accelerating with severity. c, Block precision versus demand-spike multiplier, flat and near-ceiling.

request volume, we report a volume-normalised reward over an episode of T steps, $\bar{r} = \frac{1}{T}\sum_{t=1}^{T} r_t / \max(n_t, 1)$, which divides each step reward r_t by its own pending-request count n_t before averaging.

Across both dropout and churn the cost accelerates with severity rather than rising in proportion, with 9 or 10 of every 10 seeds hurt and Wilcoxon $p \leq 0.03$. For AXIOM the dropout cost grows 0.165, 0.601, 1.710 across the three severities, and the churn cost grows 0.059, 0.200, 0.529. Every policy shows the same accelerating shape, so this is a property of the task rather than of any one design. The demand spike leaves decision quality intact on the volume-invariant block-precision metric in Fig. 7c, so its apparent reward gain is a counting effect, not a better policy.

One disruption result needed a second, independent sample to report honestly. An early pass read the independent DQN as immune to churn, since it never attends to another agent's features, but that rested on a small and non-significant ten-seed sample. A disjoint replication overturns it, with a churn cost that is significant at every severity and in fact larger than the coordinated policies', rising 0.097, 0.657, 1.223 against the earlier 0.000, 0.168, 0.640. The honest conclusion is that churn hurts every policy, and that the earlier immunity claim was a small-sample artifact. This was the single correction among four headline cross-section comparisons re-run against independent samples. The other three held with overlapping intervals, namely AXIOM over the independent DQN on reward at +1.461 then +0.767 with $p < 0.0001$ in both, AXIOM over the single-agent policy at +0.195 then +0.445, and a negligible federation cost in both.

5.5 Cluster-Size Scaling

TODO(MANOJ): Old-environment numbers. Replace with the shared-budget N=7 and N=19 scaling results, regenerate Figs 8 and 9, when the rerun finishes.

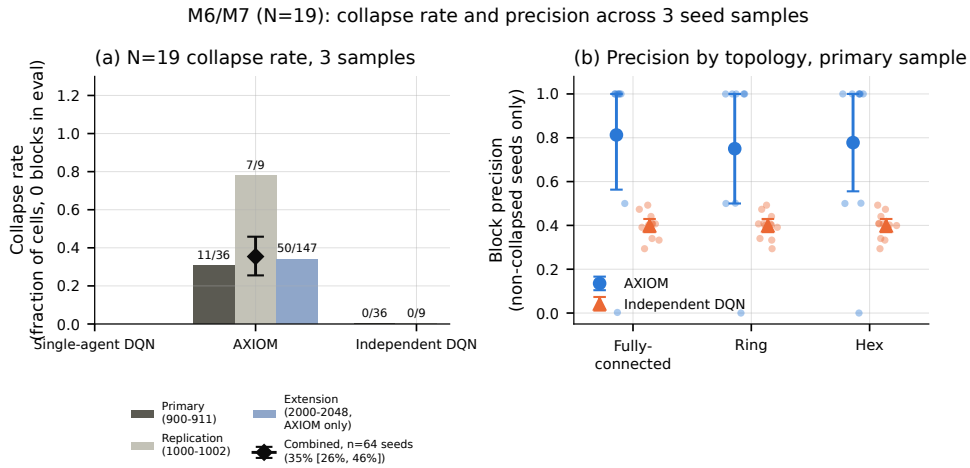


Fig. 8 N=19 cluster, per-seed evidence. a, Collapse rate pooled across topologies, single-agent DQN total and AXIOM converging to 35.4% across 64 seeds. b, Block precision by topology for non-collapsed seeds.

We grow the cluster from three gNBs to seven and nineteen under three layouts, fully-connected, ring, and hex-grid, scaling the synthetic arrivals with the cluster so per-gNB load stays roughly constant. Only AXIOM's decision head actually reads this layout, so the two DQN policies double as a check that nothing depends on topology by accident. We normalise reward per gNB with the same volume-normalised $\bar{r}$ as in Section 5.4, and we lean on block precision rather than compliance here, because compliance is combined across gNBs with a logical OR and drifts to zero as the cluster grows whatever the policy does. The main sample is twelve seeds at nineteen gNBs across every layout and policy, with further independent seeds to test generalisation.

At nineteen gNBs the per-gNB reward margins are not significant for any layout, and that is expected rather than disappointing. The single-agent policy's always-accept collapse banks the service reward on every request, while AXIOM pays a real price each time it correctly rejects low-priority mMTC traffic. The true separation therefore shows up in collapse resistance and block precision, not in raw reward.

The contrast between the three policies at scale is stark. The single-agent DQN collapses completely at nineteen gNBs, in all 45 of 45 cells across both samples and every layout, which a direct probe of its decision values confirms at the level of individual choices. AXIOM, by contrast, collapses only part of the time. The independent DQN never collapses fully, but settles into a narrow and mediocre 0.29–0.49 block-precision band that is identical across layouts, exactly as expected for a policy that never reads its neighbours.

Pooled across 64 independently drawn seeds, covering 68 of 192 cells, AXIOM's collapse rate converges to a stable $\hat{p}_{\text{collapse}} = 35.4\%$ with 95% CI [25.5%, 45.8%]. This single bootstrapped estimate replaces two smaller samples that had disagreed. A properly powered twelve-seed run at seven gNBs, with a disjoint twelve-seed replication,

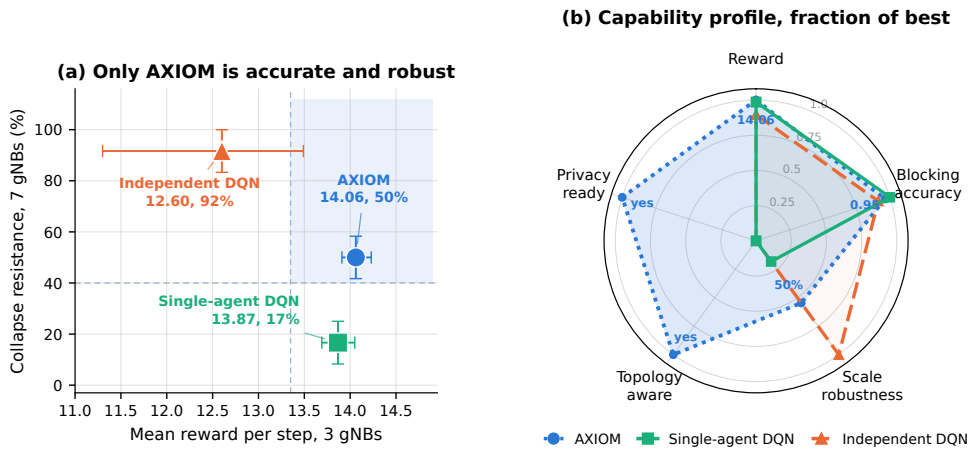


Fig. 9 AXIOM against the two baselines. a, Reward quality at three gNBs with 95% CIs against collapse resistance at seven gNBs across two 12-seed samples, where AXIOM alone is accurate and robust. b, Each axis as a fraction of the best policy on it, so AXIOM encloses the largest area and leads the two capability axes outright.

puts AXIOM at 41.7% and 58.3% collapse across the two samples. Both are statistically indistinguishable from this pooled 35.4% estimate, so this collapse-proneness is present from seven gNBs onward, not only at large scale. Over the same two samples the single-agent DQN collapses at 91.7% and 75.0%, against 0% in its earlier three-seed pilot, the sharpest single result in this extension, while the independent DQN collapses at 0.0% and 16.7%, confirming that it collapses rarely rather than never. AXIOM therefore sits between the two, far more robust than the single-agent policy it is meant to replace, and the only one of the three that stays robust while also learning a strong, topology-aware blocking rule. Fig. 9a places the three policies on one map of reward quality against scaling robustness. AXIOM alone falls in the region that is both accurate and robust, while the independent DQN trades reward away for its robustness and the single-agent policy trades robustness away for reward. The capability radar in Fig. 9b makes the same point across five axes at once, where AXIOM holds the largest enclosed area.

5.6 Live Deployment

This section reports what the real OAI testbed of Fig. 3, introduced in Section 3, can and cannot establish about the offline-trained policy. AXIOM itself is not deployed live. Its multi-gNB policy needs at least two live gNBs, and our two-gNB attempt registered both gNBs with the core but did not sustain traffic, so a live AXIOM run is left to future work. This section therefore validates the deployment pipeline that AXIOM would use, namely the E2 control path, the PRB-ratio actuation, and real-time inference of an offline-trained checkpoint. For that we run a single-agent-DQN

checkpoint, seed 900, against a real gNB, real User Equipments (UEs), and real per-slice User Datagram Protocol (UDP) traffic, at one UE per slice and again at two UEs per slice.

A capacity probe at a deliberately wide-open ceiling showed all three slices serving essentially 100% of offered demand with 0.000% loss at both loads. The frozen checkpoint's decision logic runs correctly end to end, through the real E2 control loop, with real accept and reject decisions, and its signature low-priority-only blocking of mMTC reappears at both loads, which establishes deployability end to end. Because the rig never actually runs short of capacity, the same pattern is equally consistent with the policy's trained priority bias, so we claim admission-control behaviour under contention only in the disclosed offline stress regime, not live. Two integrity issues surfaced and were fixed during these checks. A scheduler-configuration error had turned the per-slice PRB-ratio ranges into raw grants below the gNB scheduler's hard five-PRB floor, which we fixed at the root, after which direct instrumentation confirmed zero below-floor grants across more than 100,000 readings. A separate unit mismatch in the live SLA-margin diagnostic was corrected by converting backlog into a queueing-delay estimate, using Little's law, against each slice's own latency budget.

6 Conclusion

TODO(MANOJ): Rewrite the conclusion from the shared-budget results once all reruns finish.

This paper presented AXIOM, a correctness-aware graph-attention architecture for federated multi-agent slice admission control. A shared GAT encoder reads the cluster graph and feeds decentralised per-slice action heads. The design closes three gaps from our prior systematic review, namely trustworthy evaluation, privacy-preserving coordination, and validated deployment.

The architecture delivers on each of these fronts. AXIOM beats the independent-learner ablation decisively on the training objective, resists at 19 gNBs a collapse that overwhelms the single-agent baseline, and holds its reward under federation while differential privacy stays accurate up to a clear noise threshold. A live single-gNB deployment confirms that the offline-trained logic executes correctly on real hardware. These results carry honest bounds, since the multi-gNB, federated, and disruption campaigns are offline, the graph models only the gNB cluster, and the federated setting assumes a single operator.

Two safeguards keep these results trustworthy. A correctness-aware metric pair ties evaluation to the training objective, while a reproduction-versus-replication discipline tests each headline finding on an independent seed sample. Together they caught issues a standard pipeline would have reported as clean. The broader lesson is that evaluation here needs metrics tied to the objective, that architectural claims need an independent sample before they are trusted, and that a live testbed's reach must be checked rather than assumed.

Future work targets the remaining gaps from our review. The first goal is a full-scale simulation with real contention, so admission-control behaviour under genuine scarcity can be measured directly rather than in the disclosed stress regime. The second is Multi-Access Edge Computing (MEC) scaling, which adds compute-placement nodes

to the cluster graph and extends the controller to joint admission and offloading. We also plan a live multi-gNB deployment, broader privacy settings across operators, and direct Quality of Experience (QoE) inference in the reward. Together these steps move the architecture toward a scalable, privacy-aware, and user-centric controller for 5G and Beyond-5G networks.

Acknowledgements. This work was supported by the Fundamental Research Grant Scheme (FRGS) of the Ministry of Higher Education (MOHE), Malaysia, under grant FRGS/1/2024/ICT06/TAYLOR/02/1.

Declarations

The following statements should be understood to apply to this manuscript unless otherwise indicated.

- **Funding:** This work was supported by the Fundamental Research Grant Scheme (FRGS), Ministry of Higher Education (MOHE), Malaysia, under grant number FRGS/1/2024/ICT06/TAYLOR/02/1.
- **Competing interests:** The authors declare no competing interests.
- **Ethics approval and consent to participate:** Not applicable, since this study involves no human participants, animal subjects, or identifiable patient data. All reported results are either offline simulation campaigns or measurements from a laboratory radio testbed.
- **Consent for publication:** Not applicable, since this manuscript contains no personal data or identifying information about any individual.
- **Data availability:** The experiment logs, checkpoints, and campaign result files underlying this paper's findings are available from the corresponding author on reasonable request.
- **Materials availability:** Not applicable, since this study involves no physical or biological materials.
- **Code availability:** The framework and experiment scripts used to produce this paper's results, including the reproducibility manifest and analysis scripts used to independently re-derive every table and figure in this paper, are available from the corresponding author on reasonable request.
- **Author contribution:** M. P. Kunasegran: conceptualisation, methodology, software, investigation, writing (original draft). W. L. Pang: supervision, writing (review and editing). S. K. Phang: supervision, writing (review and editing).

References

[1] Abadi M, Chu A, Goodfellow I, et al (2016) Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp 308–318, https://doi.org/10.1145/2976749.2978318

[2] Ahmadi M, Moayyedi A, Sulaiman M, et al (2024) Generalizable 5G RAN/MEC slicing and admission control for reliable network operation. IEEE Trans Netw Service Manag https://doi.org/10.1109/TNSM.2024.3437217

[3] Bega D, Gramaglia M, Banchs A, et al (2020) A machine learning approach to 5G infrastructure market optimization. IEEE Transactions on Mobile Computing 19(3):498–512. https://doi.org/10.1109/TMC.2019.2896950

[4] Bouroudi A, Outtagarts A, Hadjadj-Aoul Y (2025) GNN-based multi-agent DRL for energy-efficient multi-domain 6G resource orchestration. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 4457–4462, https://doi.org/10.1109/GLOBECOM59602.2025.11432814

[5] Bun M, Steinke T (2016) Concentrated differential privacy: Simplifications, extensions, and lower bounds. In: Theory of Cryptography Conference (TCC), pp 635–658, https://doi.org/10.1007/978-3-662-53641-4_24

[6] Fatehi K, Rahmani Ghourtani M, Sonee A, et al (2026) Interpretable attention-based multi-agent PPO for latency spike resolution in 6G RAN slicing. In: Proc. IEEE Int. Conf. Commun. (ICC), https://doi.org/10.1109/ICC59461.2026.11586913

[7] Gong Y, Gong P, Jiang H, et al (2026) Intelligent resource allocation for multiaccess edge computing in 5G ultra-dense slicing network using federated multiagent DDPG. China Commun 23(1):273–289. https://doi.org/10.23919/JCC.fa.2022-0187.202601

[8] Kunasegran MP, Pang WL, Phang SK (2025) Optimising resource allocation in 5G networks: Balancing URLLC and eMBB traffic under gNB congestion. In: 2025 Multimedia University Engineering Conference (MECON), pp 1–6, https://doi.org/10.1109/MECON67253.2025.11277096

[9] Kunasegran MP, Pang WL, Phang SK (2025) SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN. In: 2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE), pp 207–212, https://doi.org/10.1109/ICRAIE65839.2025.11239148

[10] Kunasegran MP, Pang WL, Phang SK (2026) Comprehensive review of optimisation techniques for user-centric distributed network slicing in 5G networks. IEEE Access 14:114630–114648. https://doi.org/10.1109/ACCESS.2026.3716911

[11] Li T, Sahu AK, Zaheer M, et al (2020) Federated optimization in heterogeneous networks. In: Proceedings of Machine Learning and Systems (MLSys), pp 429–450

[12] McMahan B, Moore E, Ramage D, et al (2017) Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), pp

1273–1282

[13] Mohajer A, Mirzaei A, Darabi M, et al (2026) Joint SLA-aware task offloading and adaptive service orchestration with graph-attentive multi-agent reinforcement learning. IEEE Trans Netw Service Manag 23:3737–3754. https://doi.org/10.1109/TNSM.2026.3673188

[14] Polese M, Bonati L, D'Oro S, et al (2023) ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms. IEEE Transactions on Mobile Computing 22(10):5787–5800. https://doi.org/10.1109/TMC.2022.3188013

[15] Polese M, Bonati L, D'Oro S, et al (2023) Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. IEEE Communications Surveys & Tutorials 25(2):1376–1411. https://doi.org/10.1109/COMST.2023.3239220

[16] Qasim M, Imran M, Akram F, et al (2025) Transformer and graph neural network-based federated learning for cellular traffic prediction with sustainability analysis. IEEE Access 13:159649–159666. https://doi.org/10.1109/ACCESS.2025.3605380

[17] Rashid T, Samvelyan M, Schroeder de Witt C, et al (2020) Monotonic value function factorisation for deep multi-agent reinforcement learning. Journal of Machine Learning Research 21(178):1–51

[18] Raza MR, Natalino C, Öhlen P, et al (2019) Reinforcement learning for slicing in a 5G flexible RAN. Journal of Lightwave Technology 37(20):5161–5169. https://doi.org/10.1109/JLT.2019.2924345

[19] Shao Y, Li R, Hu B, et al (2021) Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network. IEEE Trans Veh Technol 70(10):10792–10803. https://doi.org/10.1109/TVT.2021.3103416

[20] Sulaiman M, Moayyedi A, Ahmadi M, et al (2023) Coordinated slicing and admission control using multi-agent deep reinforcement learning. IEEE Trans Netw Service Manag 20(2):1110–1124. https://doi.org/10.1109/TNSM.2022.3222589

[21] Tashman DH, Cherkaoui S (2026) Adversarial attacks in AI-driven RAN slicing: SLA violations and recovery. In: Proc. Int. Wireless Commun. and Mobile Computing (IWCMC), https://doi.org/10.1109/IWCMC69287.2026.11580033

[22] Tsampazi M, D'Oro S, Polese M, et al (2023) A comparative analysis of deep reinforcement learning-based xApps in O-RAN. In: Proc. IEEE Global Communications Conference (GLOBECOM), pp 1638–1643, https://doi.org/10.1109/GLOBECOM54140.2023.10437367

[23] Veličković P, Cucurull G, Casanova A, et al (2018) Graph attention networks. In: International Conference on Learning Representations (ICLR)

[24] Yasin TV, Yu CM, Wang LC (2025) Differential privacy federated edge learning-assisted for securing RAN intelligent controller in O-RAN 6G communications. In: Proc. IEEE VTS Asia Pacific Wireless Commun. Symp. (APWCS), https://doi.org/10.1109/APWCS67981.2025.11151868

[25] Yu C, Velu A, Vinitsky E, et al (2022) The surprising effectiveness of PPO in cooperative multi-agent games. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track

[26] Zhang H, Zhou H, Erol-Kantarci M (2022) Federated deep reinforcement learning for resource allocation in O-RAN slicing. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 958–963, https://doi.org/10.1109/GLOBECOM48099.2022.10001658